

Taller: Nest JS

Proyecto: Aplicación de gestión de librerías multimedia (juegos, películas y libros favoritos)

Entrega: Domingo, Octubre 11 de 2026

Objetivo

Desarrollar una aplicación backend robusta con Nest JS, que utilice PostgreSQL para la persistencia de datos y pruebas unitarias, para la gestión de librerías multimedia.

Descripción del proyecto

Actualmente, muchos usuarios llevan el registro de sus juegos, películas y libros favoritos o pendientes en notas sueltas, hojas de cálculo o aplicaciones independientes por cada tipo de medio, pero no existe una plataforma centralizada que permita organizar toda su colección, hacer seguimiento a su avance, calificar lo que consumen o compartir recomendaciones.

Por esta razón, se requiere una API que permita a los usuarios crear y administrar sus bibliotecas multimedia, registrar el estado y el progreso de cada título, organizar sus favoritos en listas personalizadas y consultar las reseñas de la comunidad, con moderación y administración según el rol.

1. Requisitos mínimos

➤ Seed (5%)

- Alimentar la base de datos con registros iniciales que permitan la realización de pruebas y el funcionamiento de la aplicación: los tres roles, un usuario de cada rol (administrador, moderador y usuario), al menos 10 géneros, al menos 5 elementos aprobados por cada tipo (juegos, películas y libros), bibliotecas de ejemplo para el usuario de prueba y al menos dos listas (una pública y una privada).
- Usar un endpoint protegido (POST /seed, solo administrador) o un script (npm run seed) que permita el cargue inicial de los datos. El seed debe poder ejecutarse más de una vez sin duplicar registros.

➤ Autenticación (7%)

- Implementar un sistema de autenticación basado en tokens JWT (Json Web Tokens), con registro e inicio de sesión con correo y contraseña. La contraseña se almacena cifrada con hash (bcrypt) y nunca se devuelve en las respuestas.
- Los usuarios deben poder iniciar sesión y cerrar sesión. Al cerrar sesión el token se revoca y deja de ser válido, aunque no haya expirado.
- Debe haber rutas protegidas que requieran autenticación: todas las rutas, excepto el registro, el inicio de sesión y la documentación de Swagger.

➤ **Autorización (8%)**

- Definir tres roles diferentes: usuario, moderador y administrador, con los permisos descritos en las secciones “Tipos de usuarios” y “Permisos por rol”.
- Establecer los permisos basados en roles (guards y decoradores de Nest JS) para restringir el acceso a ciertas rutas o funcionalidades, y validar la propiedad de los recursos (un usuario solo puede modificar lo que le pertenece).
- Los roles deben asignarse mediante un mecanismo de administración: el endpoint PATCH /users/:id/roles, disponible únicamente para administradores.

➤ **Pruebas (20%)**

- Implementar pruebas unitarias (Jest) para los servicios principales: autenticación, catálogo, biblioteca personal, listas y reseñas, incluyendo las reglas de negocio descritas en “Funcionamiento de la biblioteca”.
- Implementar pruebas de integración o extremo a extremo (supertest) para los flujos de registro, inicio y cierre de sesión, acceso a rutas protegidas y restricciones por rol (respuestas 401 y 403).

➤ **Persistencia en base de datos (10%)**

- Utilizar TypeORM para interactuar con una base de datos relacional PostgreSQL.
- Modelar las entidades, enumeraciones, tipos de datos, restricciones y relaciones descritas en la sección “Modelo de datos”.

➤ **Funcionalidades (20%)**

- Implementar en el backend las funcionalidades descritas en las secciones “Funcionamiento de la biblioteca” y “Endpoints de la API”.
- Adjuntar el archivo json de Postman que permita ejecutar las pruebas de forma manual, con una colección organizada por módulo y variables de entorno para la URL base y el token.

➤ **Informe (10%)**

- Preparar un informe detallado que describa las funcionalidades implementadas en la API.
- El informe debe incluir una descripción de cada endpoint, sus parámetros y respuestas.
- Además, debe explicar cómo se implementaron las características de autenticación, autorización y persistencia de la base de datos.

➤ **Despliegue (10%)**

- Se debe desplegar la API y la base de datos PostgreSQL en algún servicio en nube, configurando las variables de entorno (conexión a la base de datos, secreto y expiración del JWT).

- Para evitar consumir sus créditos, realice un video demostrando el despliegue del aplicativo y cierre el despliegue.
- **Swagger (5%)**
 - Agregar la documentación de Swagger en los endpoints (ruta /api/docs), incluyendo la descripción de los DTOs, los códigos de respuesta y la autenticación Bearer.
- **Github Actions (5%)**
 - Agregar un pipeline de Github Actions que compruebe, al hacer un pull request y al hacer un push a la rama main, que las pruebas funcionen correctamente (instalación de dependencias, lint, pruebas unitarias y pruebas e2e con un servicio de PostgreSQL).

2. Tipos de usuarios

La aplicación maneja tres roles almacenados en la base de datos, además del visitante que aún no ha iniciado sesión. Los permisos son acumulativos: el moderador incluye los del usuario y el administrador incluye los del moderador.

Tipo	Descripción	Cómo se obtiene
Visitante (sin sesión)	Persona no autenticada. No es un rol almacenado en la base de datos.	Solo puede registrarse, iniciar sesión y consultar la documentación de Swagger.
Usuario (USER)	Persona registrada que organiza su propia colección de juegos, películas y libros, escribe reseñas y comparte listas.	Rol por defecto al registrarse.
Moderador (MODERATOR)	Tiene todos los permisos del usuario y además revisa los elementos propuestos al catálogo y las reseñas reportadas.	Asignado por un administrador.
Administrador (ADMIN)	Tiene todos los permisos del moderador y además gestiona usuarios, roles, géneros, catálogo y el seed.	Creado en el seed; puede asignar roles a otros usuarios.

3. Tipos de permisos

Cada acción de la API se clasifica en uno de los siguientes tipos de permiso, y cada permiso tiene un alcance.

Tipo de permiso	Código	Descripción
Lectura	read	Consultar recursos (catálogo, biblioteca, listas, reseñas, estadísticas).
Creación	create	Crear recursos nuevos (entradas de biblioteca, listas, reseñas, propuestas de catálogo).

Tipo de permiso	Código	Descripción
Edición	update	Modificar recursos existentes.
Eliminación	delete	Borrar recursos.
Moderación	moderate	Aprobar o rechazar elementos propuestos; ocultar o restaurar reseñas reportadas.
Administración	manage	Gestionar usuarios, roles, géneros, catálogo completo y ejecutar el seed.

Alcance	Descripción
Propio (own)	El permiso aplica solo a los recursos cuyo dueño es el usuario autenticado. Se valida en el servicio comparando el id del usuario con el del recurso; si no coincide, responde 403.
Cualquiera (any)	El permiso aplica a los recursos de cualquier usuario. Se otorga solo mediante el rol (moderador o administrador).

4. Permisos por rol

La siguiente matriz resume qué puede hacer cada rol. “Propio” indica que solo aplica a recursos del mismo usuario; “Sí” indica que aplica a los de cualquier usuario; “No” indica que la API responde 403.

Recurso	Acción	Usuario	Moderador	Administrador
Cuenta	Ver y editar su propio perfil	Propio	Propio	Propio
Usuarios	Listar usuarios y ver el detalle de cualquiera	No	No	Sí
Usuarios	Asignar roles y activar o desactivar cuentas	No	No	Sí
Catálogo	Consultar elementos aprobados	Sí	Sí	Sí
Catálogo	Proponer un elemento personalizado (queda pendiente)	Sí	Sí	Sí
Catálogo	Editar un elemento propio mientras esté pendiente	Propio	Propio	Propio
Catálogo	Aprobar o rechazar elementos propuestos	No	Sí	Sí
Catálogo	Editar o eliminar cualquier elemento	No	No	Sí
Géneros	Consultar géneros	Sí	Sí	Sí
Géneros	Crear, editar y eliminar géneros	No	No	Sí
Biblioteca	Agregar, editar y quitar entradas propias	Propio	Propio	Propio
Biblioteca	Ver la biblioteca de otro usuario	No	No	Sí
Listas	Crear, editar y eliminar listas propias	Propio	Propio	Propio

Recurso	Acción	Usuario	Moderador	Administrador
Listas	Consultar listas públicas de otros usuarios	Sí	Sí	Sí
Reseñas	Crear, editar y eliminar reseñas propias	Propio	Propio	Propio
Reseñas	Consultar reseñas visibles y reportar una reseña ajena	Sí	Sí	Sí
Reseñas	Ver reportes; ocultar o restaurar reseñas	No	Sí	Sí
Reseñas	Eliminar cualquier reseña	No	No	Sí
Estadísticas	Consultar sus propias estadísticas e historial	Propio	Propio	Propio
Estadísticas	Consultar estadísticas globales de la plataforma	No	No	Sí
Seed	Ejecutar el cargue inicial de datos	No	No	Sí

5. Modelo de datos

Se utiliza TypeORM sobre PostgreSQL. Todas las claves primarias son uuid. En las tablas siguientes, la columna “Tipo” indica el tipo en TypeScript y el tipo en la base de datos.

Enumeraciones

Enumeración	Valores
Role	USER, MODERATOR, ADMIN
MediaType	GAME, MOVIE, BOOK
ApprovalStatus	PENDING, APPROVED, REJECTED
LibraryStatus	PENDING (por ver, jugar o leer), IN_PROGRESS, COMPLETED, DROPPED
ListVisibility	PRIVATE, PUBLIC
ReportReason	SPOILER, OFFENSIVE, SPAM, OTHER
ReportStatus	OPEN, RESOLVED, DISMISSED
ActivityAction	ENTRY_ADDED, STATUS_CHANGED, PROGRESS_UPDATED, ENTRY_COMPLETED, FAVORITE_TOGGLED, REVIEW_CREATED, LIST_CREATED

User (users)

Campo	Tipo	Restricciones y descripción
id	string / uuid	Clave primaria generada automáticamente.
email	string / varchar(120)	Obligatorio, único, con formato de correo válido.
username	string / varchar(30)	Obligatorio, único, de 3 a 30 caracteres (letras, números y guion bajo).

Campo	Tipo	Restricciones y descripción
password	string / varchar(255)	Hash bcrypt. Mínimo 8 caracteres antes de cifrar, con mayúscula, minúscula y número. Nunca se devuelve en las respuestas.
fullName	string / varchar(100)	Opcional.
role	Role / enum	Por defecto USER.
isActive	boolean / boolean	Por defecto true. Si es false no puede iniciar sesión.
createdAt, updatedAt	Date / timestamp	Generados automáticamente.

Genre (genres)

Campo	Tipo	Restricciones y descripción
id	string / uuid	Clave primaria.
name	string / varchar(50)	Obligatorio y único (por ejemplo: Acción, Drama, Fantasía).
description	string / varchar(255)	Opcional.

MediaItem (media_items): elemento del catálogo

Campo	Tipo	Restricciones y descripción
id	string / uuid	Clave primaria.
title	string / varchar(150)	Obligatorio.
type	MediaType / enum	Obligatorio: GAME, MOVIE o BOOK.
description	string / text	Sinopsis. Opcional.
releaseYear	number / integer	Obligatorio, entre 1900 y el año actual + 5.
creator	string / varchar(120)	Desarrollador (juego), director (película) o autor (libro).
coverUrl	string / varchar(500)	URL de la portada. Opcional.
platform	string / varchar(60)	Solo juegos (por ejemplo: PC, PlayStation). Opcional.
durationMinutes	number / integer	Solo películas. Mayor que 0. Es el máximo del progreso.
pages	number / integer	Solo libros. Mayor que 0. Es el máximo del progreso.
isbn	string / varchar(20)	Solo libros. Opcional.
approvalStatus	ApprovalStatus / enum	APPROVED si lo crea el administrador o el seed; PENDING si lo propone un usuario.
rejectionReason	string / varchar(255)	Obligatorio cuando se rechaza un elemento.
averageRating	number / decimal(3,1)	Promedio de las calificaciones personales (1 a 10). Por defecto 0.
ratingsCount	number / integer	Cantidad de calificaciones. Por defecto 0.

Campo	Tipo	Restricciones y descripción
createdBy	User / ManyToOne	Usuario que creó o propuso el elemento.
genres	Genre[] / ManyToMany	Al menos un género.
createdAt, updatedAt	Date / timestamp	Generados automáticamente. Restricción única: (title, type, releaseYear).

LibraryEntry (library_entries): elemento dentro de la biblioteca de un usuario

Campo	Tipo	Restricciones y descripción
id	string / uuid	Clave primaria.
user	User / ManyToOne	Dueño de la entrada. Si se elimina el usuario, se eliminan sus entradas.
medialtem	Medialtem / ManyToOne	Elemento del catálogo. Restricción única: (user, medialtem).
status	LibraryStatus / enum	Por defecto PENDING.
progress	number / integer	Por defecto 0, nunca negativo. Páginas leídas (libro), minutos vistos (película) u horas jugadas (juego).
rating	number / smallint	Calificación personal entera de 1 a 10. Nulo si no ha calificado.
isFavorite	boolean / boolean	Por defecto false.
notes	string / text	Notas privadas del usuario. Opcional.
startedAt, completedAt	Date / date	Nulos hasta que el estado lo determine (ver reglas).
addedAt, updatedAt	Date / timestamp	Generados automáticamente.

UserList (user_lists): lista personalizada

Campo	Tipo	Restricciones y descripción
id	string / uuid	Clave primaria.
name	string / varchar(80)	Obligatorio. Único por dueño.
description	string / varchar(255)	Opcional.
visibility	ListVisibility / enum	Por defecto PRIVATE.
owner	User / ManyToOne	Dueño de la lista.
items	Medialtem[] / ManyToMany	Tabla intermedia list_items. No admite elementos repetidos.
createdAt, updatedAt	Date / timestamp	Generados automáticamente.

Review (reviews)

Campo	Tipo	Restricciones y descripción
id	string / uuid	Clave primaria.
user	User / ManyToOne	Autor de la reseña.
medialtem	Medialtem / ManyToOne	Elemento reseñado. Restricción única: (user, medialtem).
title	string / varchar(100)	Opcional.
content	string / text	Obligatorio, de 10 a 2000 caracteres.
isHidden	boolean / boolean	Por defecto false. Una reseña oculta solo la ven su autor y los moderadores.
hiddenReason	string / varchar(255)	Motivo de ocultamiento. Opcional.
createdAt, updatedAt	Date / timestamp	Generados automáticamente.

ReviewReport (review_reports)

Campo	Tipo	Restricciones y descripción
id	string / uuid	Clave primaria.
review	Review / ManyToOne	Reseña reportada.
reportedBy	User / ManyToOne	Quien reporta. Restricción única: (review, reportedBy).
reason	ReportReason / enum	Obligatorio.
comment	string / varchar(255)	Opcional.
status	ReportStatus / enum	Por defecto OPEN.
resolvedBy	User / ManyToOne	Moderador que resolvió. Nulo mientras esté abierto.
createdAt, resolvedAt	Date / timestamp	resolvedAt es nulo mientras esté abierto.

ActivityLog (activity_logs): historial de actividad

Campo	Tipo	Restricciones y descripción
id	string / uuid	Clave primaria.
user	User / ManyToOne	Usuario que realizó la acción.
action	ActivityAction / enum	Tipo de acción registrada.
medialtem	Medialtem / ManyToOne	Elemento involucrado. Opcional.
metadata	object / jsonb	Datos adicionales (por ejemplo: estado anterior y nuevo). Opcional.
createdAt	Date / timestamp	Generado automáticamente.

RevokedToken (revoked_tokens): tokens cerrados con logout

Campo	Tipo	Restricciones y descripción
id	string / uuid	Clave primaria.
jti	string / varchar(64)	Identificador único del token JWT. Único.
expiresAt	Date / timestamp	Fecha de expiración del token; permite limpiar registros vencidos.
revokedAt	Date / timestamp	Fecha del cierre de sesión.

Relaciones principales

- User 1 — N LibraryEntry, UserList, Review, ReviewReport y ActivityLog.
- Medialtem 1 — N LibraryEntry y Review; Medialtem N — M Genre; Medialtem N — M UserList.
- Review 1 — N ReviewReport.
- User 1 — N Medialtem (createdBy).

6. Funcionamiento de la biblioteca

Estas reglas de negocio deben implementarse en los servicios y estar cubiertas por pruebas unitarias.

- 1. Colección unificada.** Cada usuario tiene una única biblioteca que agrupa juegos, películas y libros. El tipo de cada entrada lo determina el elemento del catálogo (Medialtem.type).
- 2. Agregar elementos.** Solo se pueden agregar elementos con approvalStatus APPROVED o elementos propuestos por el mismo usuario. Un elemento no puede agregarse dos veces a la misma biblioteca (error 409). La entrada nace con status PENDING y progress 0, salvo que se indique otro estado.
- 3. Estados.** Una entrada puede cambiar libremente entre PENDING, IN_PROGRESS, COMPLETED y DROPPED, aplicando automáticamente estas reglas: al pasar a IN_PROGRESS por primera vez se guarda startedAt; al pasar a COMPLETED se guarda completedAt y, si el elemento tiene máximo (páginas o minutos), el progreso se iguala a ese máximo; al salir de COMPLETED se borra completedAt.
- 4. Progreso.** La unidad depende del tipo: páginas leídas en libros (máximo: pages), minutos vistos en películas (máximo: durationMinutes) y horas jugadas en juegos (sin máximo). El progreso no puede ser negativo ni superar el máximo (error 400). Si el progreso es mayor que 0 y el estado es PENDING, pasa automáticamente a IN_PROGRESS; si en libros o películas alcanza el máximo, pasa automáticamente a COMPLETED.

5. **Calificación personal.** Es un número entero de 1 a 10 y solo puede asignarse cuando el estado es IN_PROGRESS, COMPLETED o DROPPED (error 400 si está PENDING). Al crear, cambiar o quitar una calificación, o al eliminar la entrada, se recalculan averageRating y ratingsCount del elemento.
6. **Favoritos.** Se marcan o desmarcan con el campo isFavorite de la entrada y se pueden filtrar en la consulta de la biblioteca.
7. **Listas personalizadas.** El usuario crea listas con nombre único y visibilidad PRIVATE o PUBLIC. Solo se pueden agregar elementos aprobados y sin repetir. Las listas públicas las pueden consultar todos los usuarios autenticados, mostrando el nombre de usuario del dueño; las privadas solo las ve su dueño.
8. **Elementos personalizados.** Si el elemento que busca el usuario no existe, puede proponerlo. Queda en estado PENDING, visible solo para su creador (que puede agregarlo a su biblioteca) hasta que un moderador lo apruebe o lo rechace con un motivo. Un elemento rechazado no aparece en el catálogo, pero la entrada ya creada en la biblioteca se conserva.
9. **Reseñas.** Un usuario puede escribir una sola reseña por elemento y solo si lo tiene en su biblioteca con estado IN_PROGRESS, COMPLETED o DROPPED. Puede editarla o eliminarla.
10. **Reportes y moderación.** Cualquier usuario puede reportar una reseña ajena con un motivo (una sola vez por reseña). La reseña sigue visible hasta que un moderador resuelve el reporte: HIDE oculta la reseña (isHidden = true) y marca el reporte como RESOLVED; DISMISS lo marca como DISMISSED y deja la reseña visible.
11. **Eliminación y desactivación.** Quitar un elemento de la biblioteca no elimina el elemento del catálogo ni las reseñas del usuario. Un administrador no elimina usuarios: los desactiva (isActive = false), lo que bloquea el inicio de sesión y conserva sus datos.
12. **Historial.** Cada acción relevante (agregar elemento, cambiar estado, actualizar progreso, completar, marcar favorito, crear reseña o lista) genera automáticamente un registro en ActivityLog, que el usuario consulta en GET /activity.
13. **Estadísticas personales.** GET /stats/me devuelve: total de entradas por tipo, total por estado, los 5 géneros más frecuentes, elementos completados por mes en los últimos 12 meses y la calificación promedio otorgada.

Filtros, orden y paginación

Las consultas de listado (catálogo, biblioteca, usuarios, listas públicas, reseñas e historial) aceptan los siguientes parámetros de consulta, según corresponda, y responden con la estructura { data: [], meta: { total, page, limit, totalPages } }.

Parámetro	Tipo	Descripción
q	string	Busca por texto en el título o el creador.
type	MediaType	Filtra por juego, película o libro.
genreId	uuid	Filtra por género.
year	integer	Filtra por año de lanzamiento.
status	LibraryStatus	Solo en /library: filtra por estado.
isFavorite	boolean	Solo en /library: filtra favoritos.
sortBy	string	Campo de orden: title, releaseYear, averageRating o addedAt (solo biblioteca).
order	ASC DESC	Sentido del orden. Por defecto ASC.
page	integer	Número de página, desde 1. Por defecto 1.
limit	integer	Elementos por página, de 1 a 50. Por defecto 10.

7. Endpoints de la API

Todas las rutas, excepto el registro y el inicio de sesión, requieren el encabezado Authorization: Bearer <token>. La columna “Rol mínimo” indica el rol más bajo con acceso.

Método	Ruta	Rol mínimo	Descripción
Autenticación			
POST	/auth/register	Visitante	Crea un usuario con rol USER. Body: email, username, password, fullName (opcional).
POST	/auth/login	Visitante	Body: email, password. Devuelve accessToken (JWT, expira en 1 hora) y los datos básicos del usuario.
POST	/auth/logout	Usuario	Revoca el token actual.
GET	/auth/me	Usuario	Devuelve el perfil del usuario autenticado.
Usuarios			
PATCH	/users/me	Usuario	Edita el perfil propio (username, fullName, password).
GET	/users	Administrador	Lista paginada. Filtros: role, isActive, q.
GET	/users/:id	Administrador	Detalle de un usuario.
PATCH	/users/:id/roles	Administrador	Body: role. Asigna un rol.
PATCH	/users/:id/status	Administrador	Body: isActive. Activa o desactiva la cuenta.
Géneros			
GET	/genres	Usuario	Lista todos los géneros.

Método	Ruta	Rol mínimo	Descripción
POST, PATCH, DELETE	/genres, /genres/:id	Administrador	Crea, edita o elimina un género. Eliminar uno en uso responde 409.
Catálogo			
GET	/media	Usuario	Elementos aprobados con filtros y paginación. Incluye los propios pendientes.
GET	/media/:id	Usuario	Detalle con géneros, promedio y cantidad de calificaciones.
POST	/media	Usuario	Propone un elemento. Queda PENDING (APPROVED si lo crea un administrador).
PATCH	/media/:id	Usuario / Administrador	El creador edita mientras esté PENDING; el administrador edita siempre.
DELETE	/media/:id	Administrador	Elimina un elemento del catálogo.
GET	/media/pending	Moderador	Lista los elementos pendientes de aprobación.
PATCH	/media/:id/approval	Moderador	Body: approvalStatus (APPROVED o REJECTED) y rejectionReason si se rechaza.
Biblioteca personal			
GET	/library	Usuario	Biblioteca propia con filtros y paginación.
POST	/library	Usuario	Body: mediaItemId, status (opcional). Agrega un elemento; 409 si ya existe.
GET	/library/:id	Usuario	Detalle de una entrada propia.
PATCH	/library/:id	Usuario	Body (todos opcionales): status, progress, rating, isFavorite, notes.
DELETE	/library/:id	Usuario	Quita la entrada de la biblioteca.
Listas			
GET	/lists	Usuario	Listas propias.
GET	/lists/public	Usuario	Listas públicas de otros usuarios (paginado).
POST	/lists	Usuario	Body: name, description, visibility.
GET	/lists/:id	Usuario	Detalle; una lista privada solo la ve su dueño.
PATCH, DELETE	/lists/:id	Usuario	Edita o elimina una lista propia.
POST	/lists/:id/items	Usuario	Body: mediaItemId. Agrega un elemento aprobado a la lista.
DELETE	/lists/:id/items/:mediaItemId	Usuario	Quita un elemento de la lista.
Reseñas y moderación			
GET	/media/:id/reviews	Usuario	Reseñas visibles del elemento (paginado).

Método	Ruta	Rol mínimo	Descripción
POST	/media/:id/reviews	Usuario	Body: title, content. Requiere tener el elemento en la biblioteca.
PATCH, DELETE	/reviews/:id	Usuario / Administrador	Edita o elimina la reseña propia; el administrador puede eliminar cualquiera.
POST	/reviews/:id/report	Usuario	Body: reason, comment. 409 si ya la reportó.
GET	/reports	Moderador	Reportes filtrados por status.
PATCH	/reports/:id/resolve	Moderador	Body: action (HIDE o DISMISS) y reason si se oculta.
Estadísticas, historial y seed			
GET	/stats/me	Usuario	Estadísticas propias (ver “Funcionamiento de la biblioteca”).
GET	/stats/global	Administrador	Totales de usuarios, elementos por tipo, entradas y reseñas.
GET	/activity	Usuario	Historial propio, paginado, más reciente primero.
POST	/seed	Administrador	Ejecuta el cargue inicial de datos.

Códigos de respuesta

Código	Uso
200 / 201 / 204	Operación exitosa / recurso creado / exitosa sin contenido.
400	Datos inválidos: falla la validación del DTO o una regla de negocio (progreso, calificación).
401	Falta el token, es inválido, expiró o fue revocado.
403	El rol o la propiedad del recurso no permite la acción.
404	El recurso no existe o no es visible para el usuario.
409	Conflicto: duplicado (correo, entrada de biblioteca, reporte) o recurso en uso.

8. Requisitos técnicos

- Arquitectura modular de Nest JS: módulos auth, users, genres, media, library, lists, reviews, reports, stats, activity y seed.
- Validación de entradas con DTOs, class-validator y ValidationPipe global con whitelist activado.
- Configuración mediante variables de entorno (ConfigModule): conexión a PostgreSQL, secreto y expiración del JWT. El archivo .env no se sube al repositorio; se incluye un .env.example.

- Contraseñas cifradas con bcrypt y nunca expuestas en las respuestas; tokens JWT con identificador único (jti) para poder revocarlos.
- Manejo consistente de errores con las excepciones HTTP de Nest JS y los códigos de la sección anterior.

9. Entrega

- Los estudiantes deben presentar el código fuente del proyecto junto con un README que incluya instrucciones para ejecutar la aplicación y probar cada funcionalidad.
- Se revisarán commits para determinar el nivel de participación de los estudiantes.
- Además, se debe proporcionar un informe detallado que describa las funcionalidades implementadas en la API, explicando cómo se implementaron las características de autenticación, autorización y persistencia en la base de datos. Así como la ejecución de las pruebas.